

SCOPIING

STATICO

Costruisce la tabella statica

DINAMICO

CRT (Tabella Centrale di Riferimenti)

⇒ mantiene aggiornata l'associazione tra oggetto e ambiente



Shallow Access

CRT

→ Un array di elementi indicizzati sul nome delle variabili

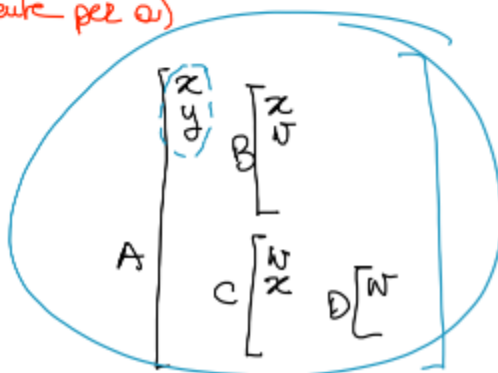
ambienti in cui x è definito e che sono ancora attivi

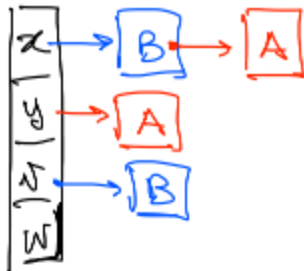
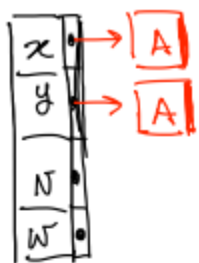
Ogni entry è una lista linkata di elementi, ogni elemento "descrive" un ambiente in cui la variabile è definita



⇒ tempo di accesso costante

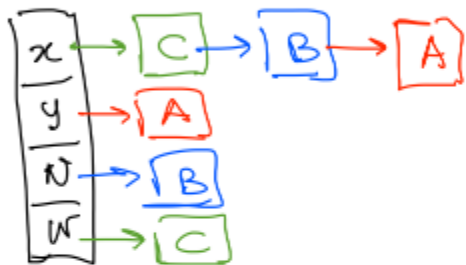
$A \rightarrow B \rightarrow C \rightarrow D$



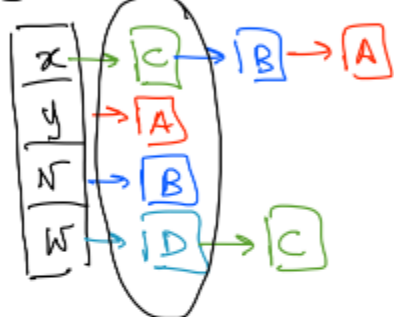


↖ Esecuzione di **A**
 ↗ Esecuzione di **B**

Esecuzione di **C**



Esecuzione di **D**



Esempio

```
{ int b;
```

```
void A() {
```

```
    int x=0, y=0, z=3;
```

```
    void B() {
```

```
        int x=3, w=3;
```

```
        → x = y + z;
```

```
        {
```

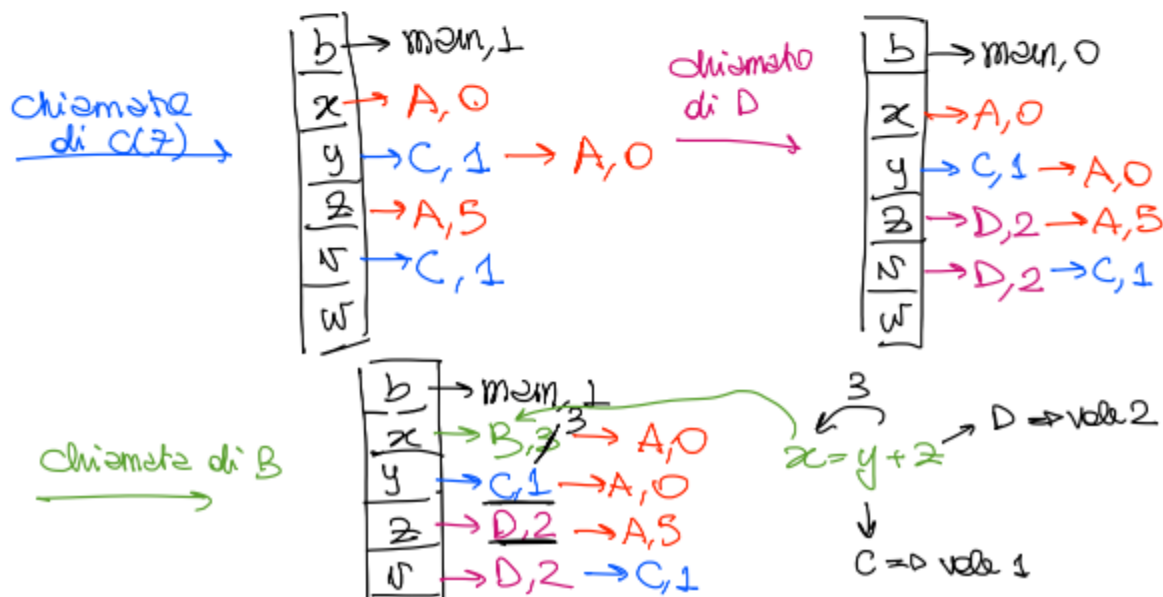
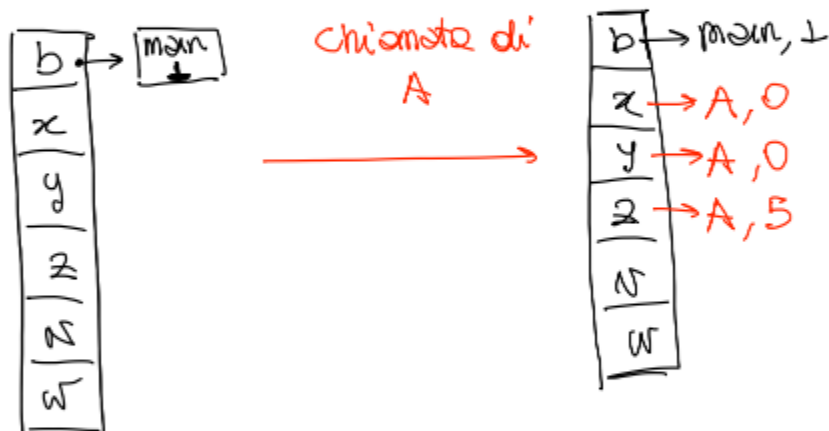
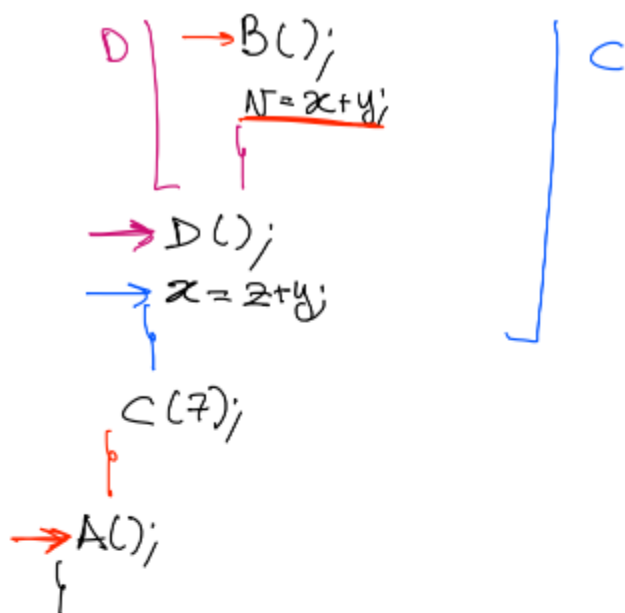
```
        void C(int q) {
```

```
            int y=1, N=1;
```

```
            void D() {
```

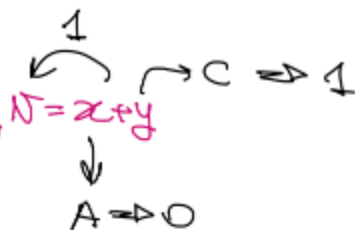
}

}



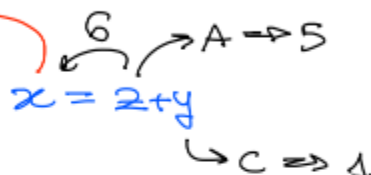
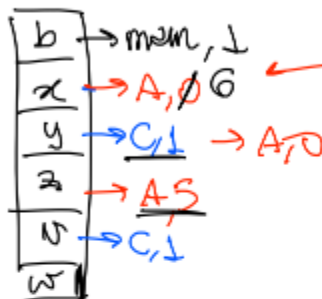
Termina B

Siamo in D



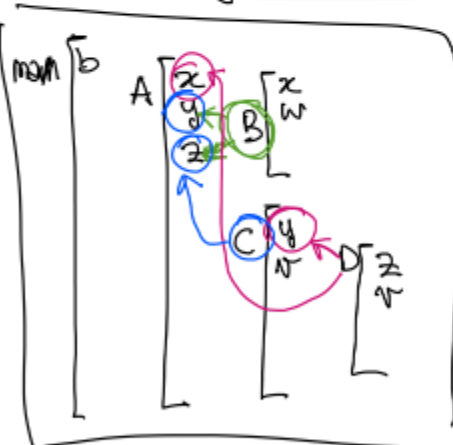
Termina D

Siamo in C



Il resto del codice termina la chiamata e distrugge
via via le liste della CFI

Scoping statico



$$\text{Sd}(\text{main}) = 0$$

$$\text{Sd}(A) = 1$$

$$\text{Sd}(C) = \text{Sd}(B) = 2$$

$$\text{Sd}(D) = 3$$

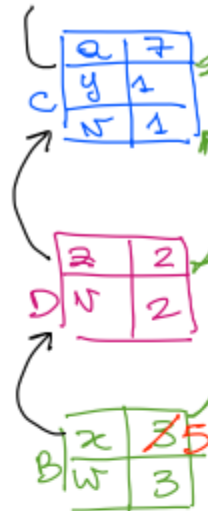
Esecuzione main



Chiamata di A

Link statico : $k_A = \text{Sd}(\text{main}) - \text{Sd}(A) + 1$

$$= 0 - 1 + 1 = 0$$



Chiamata di **C(7)**

$$\text{Link stackco: } K_c = \text{Sd}(A) - \text{Sd}(C) + 1 \\ = 1 - 2 + 1 = 0$$

$$\text{LS}(C) = A$$

Chiamata di **D**

$$\text{Link stackco: } K_D = \text{Sd}(C) - \text{Sd}(D) + 1 \\ = 2 - 3 + 1 = 0 \\ \text{LS}(D) = C$$

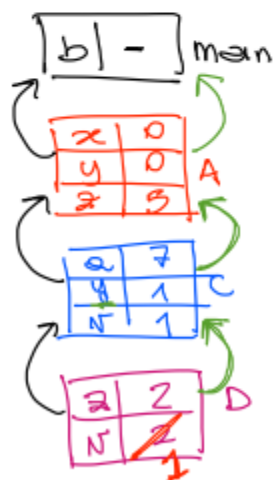
Chiamata di **B**

$$\text{Link stackco: } K_B = \text{Sd}(D) - \text{Sd}(B) + 1 \\ = 3 - 2 + 1 = 2$$

$$\text{LS}(B) = \text{LS}(\text{LS}(D)) = A$$

Esecuzione di **B**: $x = y + z \rightarrow N_z = \text{Sd}(B) - \text{Sd}(A) = 2 - 1 = 1 \rightarrow 2$
 $\downarrow$
 locale $\rightarrow N_y = \text{Sd}(B) - \text{Sd}(A) = 2 - 1 = 1 \rightarrow 2$
 $\rightarrow A \neq 0$

Terminazione di **B** $\rightarrow$ Si elimina l'RdA per B

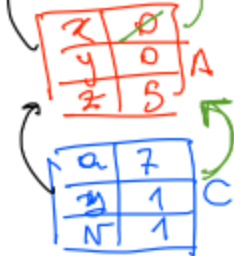


Esecuzione di **D**

$$N = x + y \rightarrow N_y = \text{Sd}(D) - \text{Sd}(C) = 2 - 1 = 1 \rightarrow 1 \\ \downarrow \\ \text{locale} \rightarrow N_x = \text{Sd}(A) - \text{Sd}(A) = 0 \rightarrow 0$$

Terminazione di **D** $\rightarrow$ Elimina RdA





Esecuzione di C:

$$x = z + y \quad \text{label} = 1$$

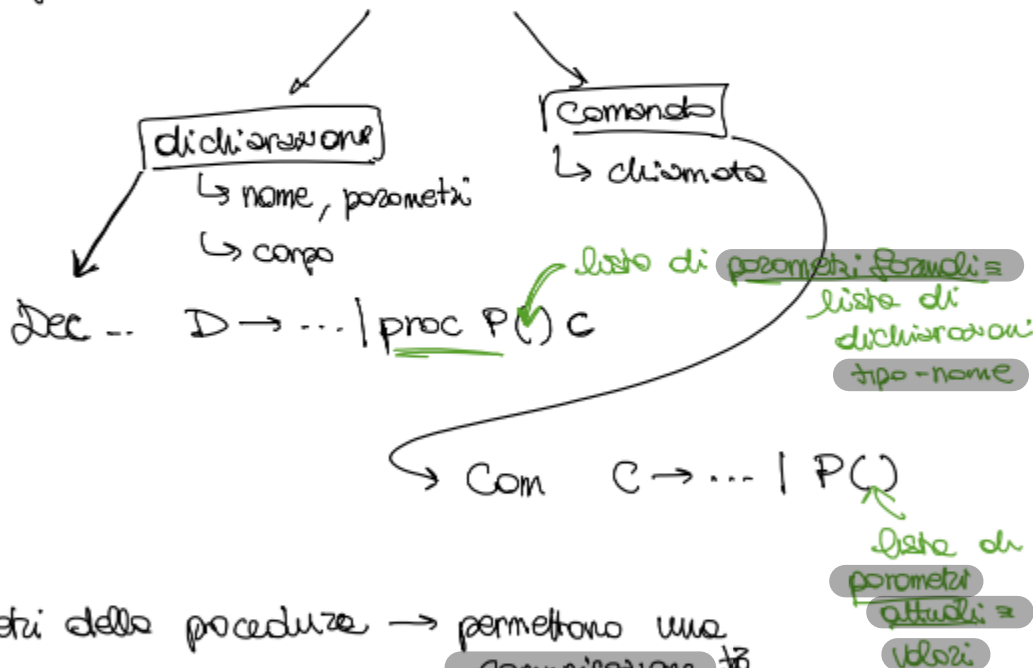
$$N_z = \text{Sd}(C) - \text{Sd}(A)$$

$$= N_x$$

$$= 1$$

Terminazione di C e di A e del main eliminano tutto dallo stack.

Nei linguaggi di programmazione per usare le procedure



Parametri delle procedure $\rightarrow$ permettono una comunicazione tra chiamante e chiamato

Dichiarazione: int f (int n) return m+1

nome
parametro formale
tipo del valore di ritorno
e-value (locazione)

Uso/chiamata: x = f(3) x = f(x+3)

r-value

Direzione della comunicazione

In-mode

Chiamante $\Rightarrow$ Chiamato

ie parametro formale e
ambiente locale e riceve

Out-mode

Chiamato $\Rightarrow$ Chiamante

ie valore del parametro attuale

Inout-mode

Chiamato $\Leftrightarrow$ Chiamante

Modello concettuale di passaggio

spostiamo fisicamente
il valore

spostiamo l'accesso
al valore